Lab Assignment 5

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC

Submitted By:

Ryan kapoor

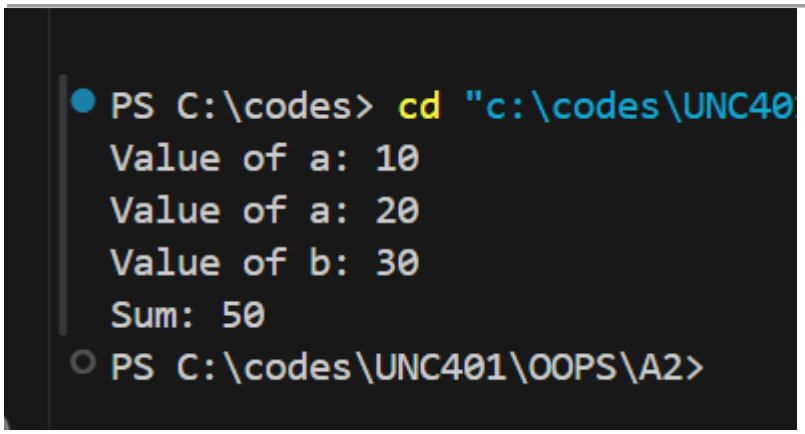
Roll No. 1024150188

Batch: 2O23

Q1

Write a simple base class, then a derived class and use objects of both of them in the main function. It will be a simple illustration of inheritance.

```
#include <iostream>
using namespace std;

class Base {
public:
    int a;

    void setA(int x) {
        a = x;
    }

    void showA() {
        cout << "Value of a: " << a << endl;
    }
};

class Derived : public Base {
public:
    int b;

    void setB(int y) {
        b = y;
    }

    void showB() {
        cout << "Value of b: " << b << endl;
    }

    void display() {
        cout << "Sum: " << a + b << endl;
    }
};

int main() {

    Base obj1;
    obj1.setA(10);
    obj1.showA();

    Derived obj2;
    obj2.setA(20);
    obj2.setB(30);
    obj2.showA();
    obj2.showB();
    obj2.display();

    return 0;
}
```

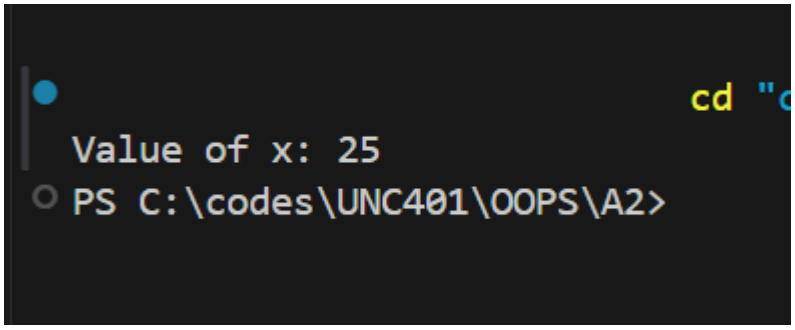


Q2

Practice protected access specifier in inheritance. In the base class declare a variable which is protected and access it in the derived class.

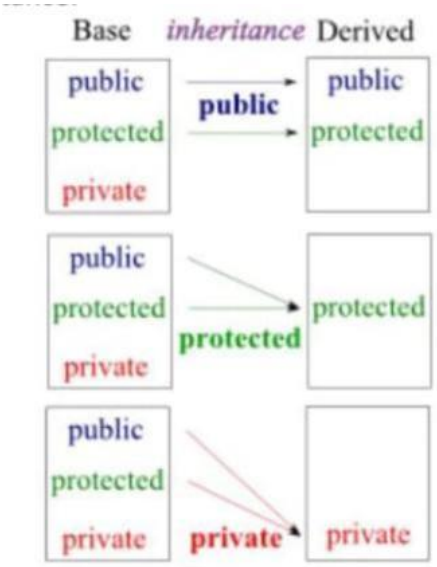
```
#include <iostream>

using namespace std;


class Base {

protected:

    int x;

};


class Derived : public Base {

public:

    void setValue(int a) {

        x = a;

    }
```

```
    void display() {
        cout << "Value of x: " << x << endl;
    }
};

int main() {

    Derived obj;
    obj.setValue(25);
    obj.display();

    return 0;
}
```



Q3

Most of the time we use public mode of inheritance, for example class Derived : public Base{}. Try protected and private access modifiers to understand the difference of various modes of inheritance.



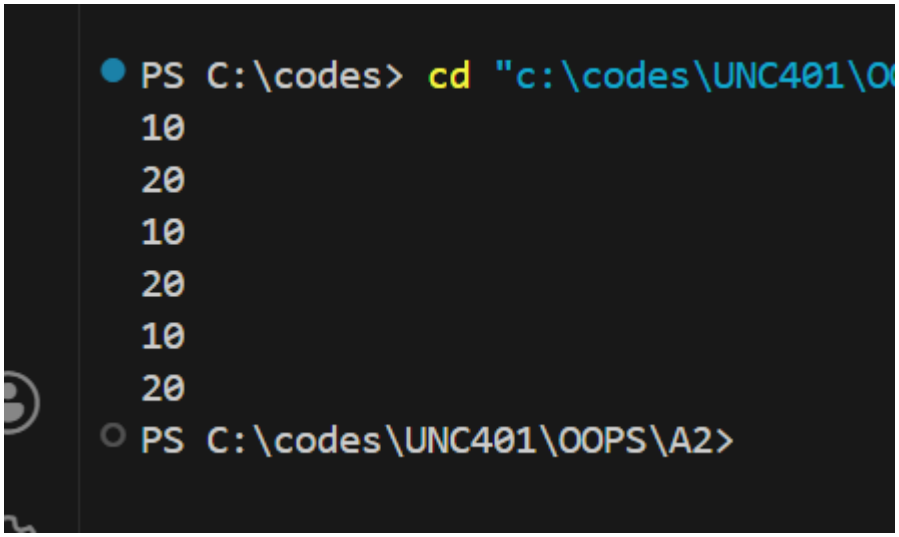

```
#include <iostream>
using namespace std;

class Base {
public:
   int a = 10;
protected:
   int b = 20;
private:
   int c = 30;
};

class PublicDerived : public Base {
public:
   void show() {
      cout << a << endl;
      cout << b << endl;
   }
};

class ProtectedDerived : protected Base {
public:
```

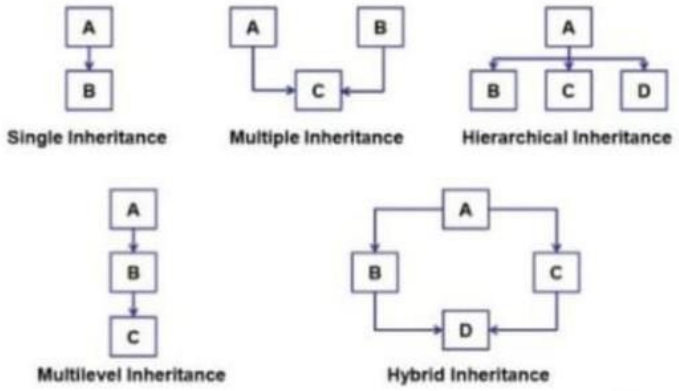
```
    void show() {
        cout << a << endl;
        cout << b << endl;
    }
};

class PrivateDerived : private Base {
public:
    void show() {
        cout << a << endl;
        cout << b << endl;
    }
};

int main() {

    PublicDerived obj1;
    obj1.show();
    ProtectedDerived obj2;
    obj2.show();
    PrivateDerived obj3;
    obj3.show();
return 0;
}
```

Q4

Various types of inheritances are as shown below. Write small C++ codes for each inheritance type:

- Single Inheritance
- Multiple Inheritance
- Hierarchical Inheritance
- Multilevel Inheritance
- Hybrid Inheritance


PS C:\codes> cd "c:\codes\UNC401\O
10
20
10
20
10
20
PS C:\codes\UNC401\OOPS\A2>



Single Inheritance:

```cpp
#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "This is class A\n";
    }
};

class B : public A {
public:
    void showB() {
        cout << "This is class B (Derived from A)\n";
    }
};

int main() {
    B obj;
    obj.showA();
    obj.showB();

    return 0;
}
```

```
This is class A
This is class B (Derived from A)


=== Code Execution Successful ===
```

Multiple Inheritance:

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void showA() {
        cout << "Class A\n";
    }
};
class B {
public:
    void showB() {
        cout << "Class B\n";
    }
};
class C : public A, public B {
public:
    void showC() {
        cout << "Class C (Derived from A and B)\n";
    }
};
int main() {
    C obj;
    obj.showA();
    obj.showB();
    obj.showC();

    return 0;
}
```

```
Class A
Class B
Class C (Derived from A and B)


=== Code Execution Successful ===
```

Hierarchical Inheritance:

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void showA() {
        cout << "This is base class A\n";
    }
};
class B : public A {
public:
    void showB() {
        cout << "This is class B\n";
    }
};
class C : public A {
public:
    void showC() {
        cout << "This is class C\n";
    }
};
int main() {
    B obj1;
    C obj2;
    obj1.showA();
    obj1.showB();
    obj2.showA();
    obj2.showC();
    return 0;
}
```

```
This is base class A
This is class B
This is base class A
This is class C


=== Code Execution Successful ===
```

Multilevel Inheritance:

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void showA() {
        cout << "Class A\n";
    }
};
class B : public A {
public:
    void showB() {
        cout << "Class B (Derived from A)\n";
    }
};
class C : public B {
public:
    void showC() {
        cout << "Class C (Derived from B)\n";
    }
};
int main() {
    C obj;
    obj.showA();
    obj.showB();
    obj.showC();
    return 0;
}
```

```
Class A
Class B (Derived from A)
Class C (Derived from B)


=== Code Execution Successful ===
```

Hybrid Inheritance:

```cpp
#include <iostream>
using namespace std;
class A {
public:
    void show() {
        cout << "Class A\n";
    }
};
class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {
public:
    void display() {
        cout << "Class D (Hybrid Inheritance)\n";
    }
};
int main() {
    D obj;
    obj.show();
    obj.display();
    return 0;
}
```

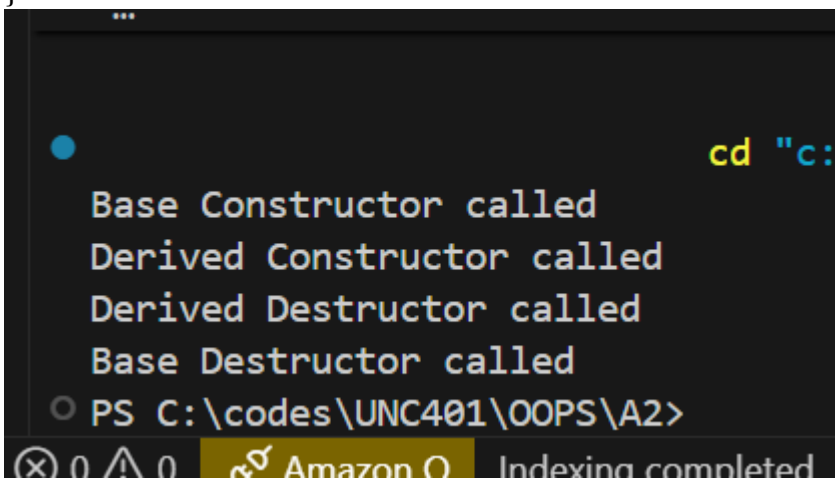
```
Class A
Class D (Hybrid Inheritance)


=== Code Execution Successful ===
```

Q5

How can you use constructors and destructors in C++ inheritance? Write a program to illustrate.

```cpp
#include <iostream>
using namespace std;

class Base {
public:
    Base() {
        cout << "Base Constructor called" << endl;
    }

    ~Base() {
        cout << "Base Destructor called" << endl;
    }
};

class Derived : public Base {
public:
    Derived() {
        cout << "Derived Constructor called" << endl;
    }

    ~Derived() {
        cout << "Derived Destructor called" << endl;
    }
};

int main() {

    Derived obj;

    return 0;
```

}

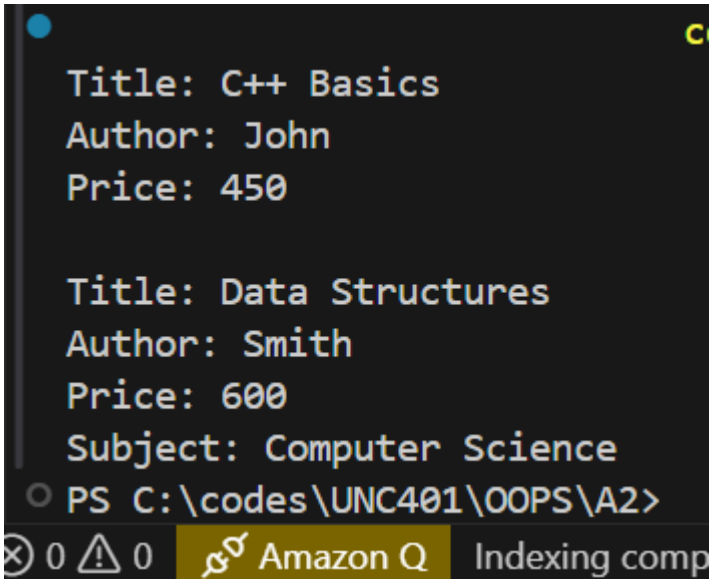

cd "c:
Base Constructor called
Derived Constructor called
Derived Destructor called
Base Destructor called
PS C:\codes\UNC401\OOPS\A2>
0 0
Amazon Q
Indexing completed

Q6

Implement a base class Book with attributes *title, author, and price*. Then, create a derived class Textbook that inherits from Book and adds a new attribute *subject*. Demonstrate how single inheritance is used to manage the data for general books and textbooks.

```
#include <iostream>

using namespace std;


class Book {

public:

    string title;

    string author;

    float price;


    void setBook(string t, string a, float p) {

        title = t;

        author = a;

        price = p;

    }


    void showBook() {

        cout << "Title: " << title << endl;

        cout << "Author: " << author << endl;

        cout << "Price: " << price << endl;

    }

};


class Textbook : public Book {

public:

    string subject;


    void setTextbook(string t, string a, float p, string s) {

        title = t;
```

```
        author = a;
        price = p;
        subject = s;
    }

    void showTextbook() {
        cout << "Title: " << title << endl;
        cout << "Author: " << author << endl;
        cout << "Price: " << price << endl;
        cout << "Subject: " << subject << endl;
    }
};

int main() {

    Book b1;
    b1.setBook("C++ Basics", "John", 450);
    b1.showBook();

    cout << endl;

    Textbook t1;
    t1.setTextbook("Data Structures", "Smith", 600, "Computer Science");
    t1.showTextbook();

    return 0;
}
```



Q7

A software company is creating a program to simulate a car's dashboard. The dashboard needs to display speed, fuel level, and temperature.

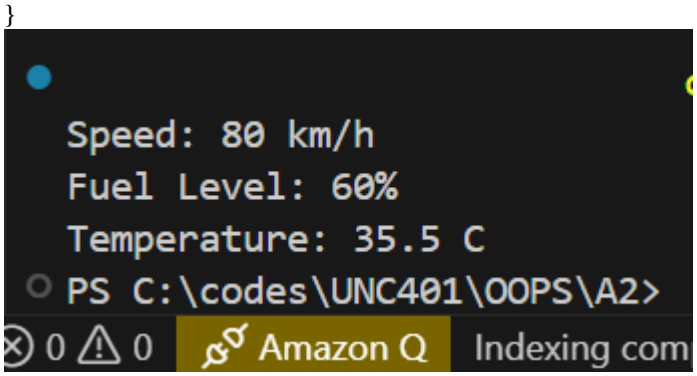
- **The speed is controlled by a Speedometer class**
- **The fuel level by a FuelGauge class**
- **The temperature by a Thermometer class**

Implement the three classes: Speedometer, FuelGauge, and Thermometer, each with relevant attributes and methods. Then, create a CarDashboard class that inherits from all three classes to display the combined information on the dashboard.

Demonstrate how multiple inheritance is used to build this class.

```
#include <iostream>
using namespace std;

class Speedometer {
public:
    int speed;

    void setSpeed(int s) {
        speed = s;
    }

    void showSpeed() {
        cout << "Speed: " << speed << " km/h" << endl;
    }
};

class FuelGauge {
public:
    int fuel;

    void setFuel(int f) {
        fuel = f;
    }
```

```
    void showFuel() {
        cout << "Fuel Level: " << fuel << "%" << endl;
    }
};

class Thermometer {
public:
    float temperature;

    void setTemperature(float t) {
        temperature = t;
    }

    void showTemperature() {
        cout << "Temperature: " << temperature << " C" << endl;
    }
};

class CarDashboard : public Speedometer, public FuelGauge, public Thermometer {
public:
    void displayDashboard() {
        showSpeed();
        showFuel();
        showTemperature();
    }
};

int main() {

    CarDashboard car;

    car.setSpeed(80);
    car.setFuel(60);
    car.setTemperature(35.5);

    car.displayDashboard();

    return 0;
}
```



Q8 You are tasked with creating a system for a library that tracks different types of users. The system needs to handle general user information such as *name, ID, and contact details*.

There are two specific types of users:

- **Student**
- **Teacher**

Each type of user has additional attributes:

- **Students → *grade level***
- **Teachers → *department***

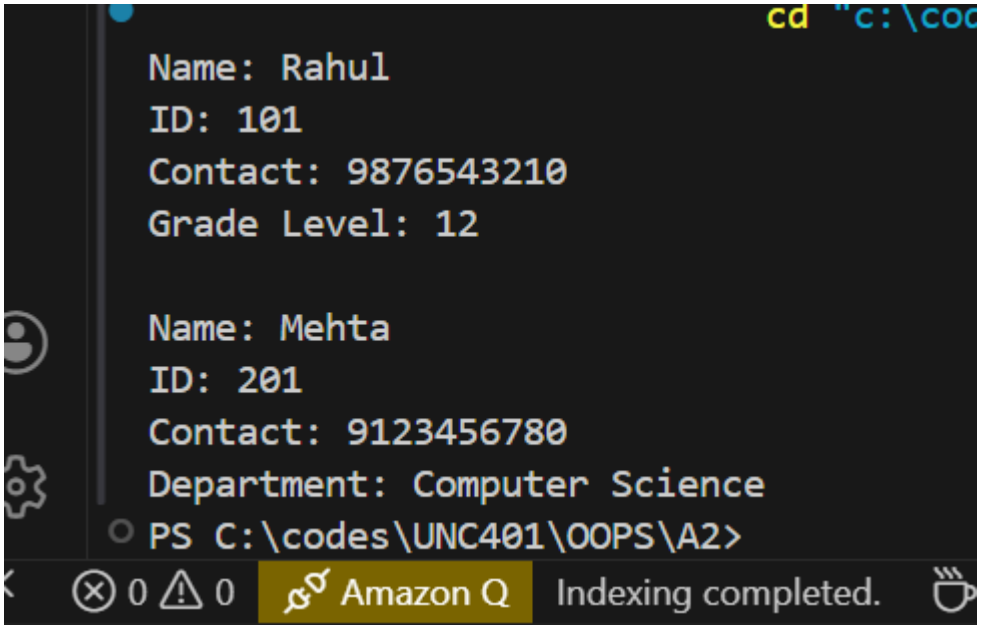
Implement a base class LibraryUser with general attributes. Then, create two derived clases Student and Teacher that inherit from LibraryUser and add their own specific attributes.

Demonstrate how hierarchical inheritance is applied in this scenario.

```
#include <iostream>
using namespace std;

class LibraryUser {
public:
   string name;
   int id;
   string contact;

   void setUser(string n, int i, string c) {
      name = n;
      id = i;
      contact = c;
   }

   void showUser() {
      cout << "Name: " << name << endl;
      cout << "ID: " << id << endl;
      cout << "Contact: " << contact << endl;
   }
};

class Student : public LibraryUser {
public:
```

```
    int grade;

    void setStudent(string n, int i, string c, int g) {
        name = n;
        id = i;
        contact = c;
        grade = g;
    }

    void showStudent() {
        showUser();
        cout << "Grade Level: " << grade << endl;
    }
};

class Teacher : public LibraryUser {
public:
    string department;

    void setTeacher(string n, int i, string c, string d) {
        name = n;
        id = i;
        contact = c;
        department = d;
    }

    void showTeacher() {
        showUser();
        cout << "Department: " << department << endl;
    }
};

int main() {

    Student s1;
    s1.setStudent("Rahul", 101, "9876543210", 12);
    s1.showStudent();

    cout << endl;
```

```
    Teacher t1;
    t1.setTeacher("Mehta", 201, "9123456780", "Computer Science");
    t1.showTeacher();

    return 0;
}
```



Q9

A logistics company needs a software system to manage its vehicle fleet.

- **All vehicles share common attributes like *make, model, and year***
- **Trucks have additional attribute: *load_capacity***
- **Refrigerated trucks have an extra attribute: *temperature_control***

Implement:

- **Base class Vehicle**
- **Derived class Truck**
- **Derived class RefrigeratedTruck (inherits from Truck)**

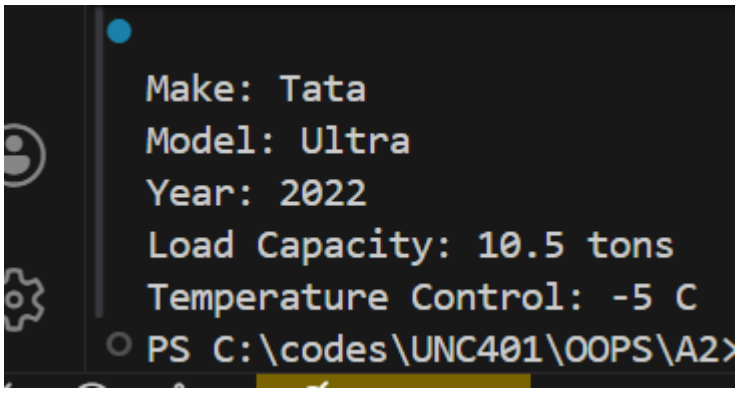
Demonstrate how multilevel inheritance works in this case.

```
#include <iostream>

using namespace std;
```

```
class Vehicle {
public:
    string make;
    string model;
    int year;

    void setVehicle(string mk, string md, int yr) {
        make = mk;
        model = md;
        year = yr;
    }

    void showVehicle() {
        cout << "Make: " << make << endl;
        cout << "Model: " << model << endl;
        cout << "Year: " << year << endl;
    }
};

class Truck : public Vehicle {
public:
    float load_capacity;

    void setTruck(string mk, string md, int yr, float lc) {
        make = mk;
```

```
        model = md;
        year = yr;
        load_capacity = lc;
    }

    void showTruck() {
        showVehicle();
        cout << "Load Capacity: " << load_capacity << " tons" << endl;
    }
};

class RefrigeratedTruck : public Truck {
public:
    float temperature_control;

    void setRefrigeratedTruck(string mk, string md, int yr, float lc, float tc) {
        make = mk;
        model = md;
        year = yr;
        load_capacity = lc;
        temperature_control = tc;
    }

    void showRefrigeratedTruck() {
        showTruck();
```

```
        cout << "Temperature Control: " << temperature_control << " C" <<
       endl;

    }

};


int main() {


    RefrigeratedTruck r1;

    r1.setRefrigeratedTruck("Tata", "Ultra", 2022, 10.5, -5.0);

    r1.showRefrigeratedTruck();


    return 0;

      }
```



Q10

You are developing a software system for an academic institution. The institution has various roles like Person, Staff, and Student.

- **A Person has general attributes like *name and address***
- **Staff (inherits Person) → attributes: *employee_id, department***
- **Student (inherits Person) → attributes: *student_id, grade***

Some Staff members are also Students (e.g., teaching assistants), so they need to inherit from both classes.

Implement:

- **Base class Person**
- **Derived classes Staff and Student**
- **A class TeachingAssistant that inherits from both Staff and Student**

Demonstrate how hybrid inheritance is applied and managed in this scenario.

```
#include <iostream>
using namespace std;

class Person {
public:
   string name;
   string address;

   void setPerson(string n, string a) {
      name = n;
      address = a;
   }

   void showPerson() {
      cout << "Name: " << name << endl;
      cout << "Address: " << address << endl;
   }
};

class Staff : virtual public Person {
public:
   int employee_id;
   string department;

   void setStaff(int id, string dept) {
      employee_id = id;
      department = dept;
   }

   void showStaff() {
      cout << "Employee ID: " << employee_id << endl;
      cout << "Department: " << department << endl;
   }
```

```
};

class Student : virtual public Person {
public:
    int student_id;
    string grade;

    void setStudent(int id, string g) {
        student_id = id;
        grade = g;
    }

    void showStudent() {
        cout << "Student ID: " << student_id << endl;
        cout << "Grade: " << grade << endl;
    }
};

class TeachingAssistant : public Staff, public Student {
public:
    void display() {
        showPerson();
        showStaff();
        showStudent();
    }
};

int main() {

    TeachingAssistant ta;

    ta.setPerson("Aman", "Delhi");
    ta.setStaff(101, "Computer Science");
    ta.setStudent(202, "A");

    ta.display();

    return 0;
}
```